

Outfit Lab · demo storyboard

One real class run, 4:50 long: 3 labs of 2 students on one laptop. The teacher presses only Next through the 19 scenes of the 7 steps.

Ana (Lab 1) and Ben (Lab 2) click the real student page; every button runs real Git. The times and IDs on screen are this run's own.

Video: demo/DEMO.mp4.

0:00 OUTFIT LAB A real class run: 3 labs of 2 students. The teacher presses only Next. Every button runs real Git.

0:04 JOIN Students open the link and type a name. The app puts them in a lab.

0:13 STEP 0 · CHAOS Each lab edits one shared outfit, live. Nothing is saved. `no Git yet`

0:20 STEP 0 · CHAOS Nobody can say who changed what. Students answer in the app: what rule would fix it?

0:24 STEP 1 · SAVE Everyone changes one part and presses Save card. Each save is a commit, with a name and a time. `git commit`

0:32 STEP 1 · SAVE Click a card: the commit object Git stored. A tree, the parent, and an author and a committer, each with a time. `git cat-file -p`

0:41 STEP 1 · SAVE One technical card per tool. Git stores the name and clock your laptop gives it. It checks neither. `git commit`

0:50 STEP 2 · TWO IDEAS Pair A builds fancy and Pair B builds sporty, each on its own sticky note. Both change TOP. `git switch -c`

1:07 STEP 2 · TWO IDEAS A branch is a tiny file that holds one commit ID. Making one copies nothing. `git branch`

1:12 STEP 3 · COMBINE One person per lab merges both ideas into main. `git merge`

1:15 STEP 3 · COMBINE Merge fancy into main: main only slides forward. A fast-forward makes no new card. `git merge`

1:20 STEP 3 · COMBINE Delete the fancy note. Git allows it because main already has its cards. `git branch -d fancy`

1:26 STEP 3 · COMBINE Merge sporty: HAT and SHOES combine by themselves. TOP changed on both sides, so a person picks. `git merge`

1:35 STEP 3 · COMBINE Finish merge writes a merge card with two parents. `git commit`

1:45 STEP 3 · COMBINE Which cards were made on fancy? No way to tell: a commit does not record its branch. `git merge`

1:50 BREAK A 4-minute break. A lab that is behind can still finish Step 3.

1:55 STEP 4 · SHARE Next puts Lab 1's outfit on the Wall, the class's shared copy. Every lab is now a full copy of it. `git clone`

1:59 STEP 4 · SHARE Each lab makes one change and saves. Lab 3 sends first: the Wall moves forward. `git push`

2:05 STEP 4 · SHARE Ana's send is refused: the Wall has a card her main lacks. A push may only move the Wall forward. `git push`

2:08 STEP 4 · SHARE Lab 1's way is Combine: fetch the Wall's cards, write a merge card with two parents, then send. `git pull --no-rebase`

2:17 STEP 4 · SHARE Ben's send is refused too. Lab 2's way is Replay on top. `git push`

2:20 STEP 4 · SHARE Replay on top: Git writes a new card with the same change, on the Wall's newest card. New parent, new ID. `git pull --rebase`

2:26 STEP 4 · SHARE Same author and author time, a new committer time. The original card is only in the safety diary. `git rebase`

2:38 STEP 4 · SHARE On the Wall: Lab 1's merge card has two parents. Lab 2's copy is a straight line with a new ID. `git push · git rebase`

2:46 STEP 4 · SHARE The teacher sees each change's path to main, with this class's real times: made → on the Wall. `code velocity`

2:53 STEP 4 · SHARE Ben's lab: its original card is not on the Wall. The path starts at a copy. `git rebase`

3:04 STEP 5 · UNDO Next: the Intern pushes a 🤖 card to the Wall. Every lab gets it, as a fast-forward. `git pull`

3:09 STEP 5 · UNDO Lab 2 tries Move my note back: main points again at the card before 🤖. `git reset --hard`

3:15 STEP 5 · UNDO The Wall refuses: it still has the 🤖 card, and a push may only move it forward. `git push`

3:19 STEP 5 · UNDO The safety diary lists every place main has been. `git reflog main`

3:24 STEP 5 · UNDO Get & combine brings the 🤖 card back: the Wall still had it. `git pull`

3:29 STEP 5 · UNDO Undo this card adds a fix card on top. History only grows, so it sends like any card. `git revert`

3:36 STEP 5 · UNDO Ana undoes it too. Her send is refused, so she combines the two fix cards (no conflict) and sends. `git revert`

3:46 STEP 5 · UNDO revert is safe on shared history. reset only if nobody else has those cards. `git revert · git reset`

3:51 STEP 6 · CLEAN UP Next asks the Wall who first added the 🦶 boots. It answers: Raj (Lab 1), card c7186b4. `git log`

3:57 STEP 6 · CLEAN UP The boss lab squashes every card after Start into one new card, and forces the Wall onto it. `squash + git push --force`

4:09 STEP 6 · CLEAN UP Asked again, the Wall says: not found. Only the labs that kept the old cards still know. `git push --force`

4:18 THE PAPER Code velocity from this class's own times. After the squash, the Wall has none of these cards. `Just et al., ISSRE 2016`

4:30 EXIT QUESTION On their own: a password reached the Wall and two labs pulled. Does revert remove it?

4:35 WRAP My Git in 7 lines: one takeaway per step, in each student's own words.

4:46 WRAP 19 scenes, 7 steps. The teacher pressed only Next, and every click ran real Git.

Outfit Lab - Teacher2 people - 2 online

Teacher consoleJoin QRStart presenting

SCENE 1 OF 19 - PLAN 000

Join

0:05 / 3:00 - on plan

SAYNo Git lecture today. You'll hit seven problems, and your lab invents a fix for each. Every button runs real Git.

DOWait until the count matches the room. The app balances the labs. Labs lock at Step 1.

Next: Step 0: Everyone, one outfitBack

Lab 1Nobody yet

Lab 21 person

Lab 31 person

ON THE PROJECTOR NOW

ADDRESSlocalhost:3310

Open the link and type your name. The app picks your lab.

Lab 1Nobody yet

Lab 2Mei

Lab 3Kim

NEXT

Step 0: Everyone, one outfit

Your lab shares one outfit. Change any part, any time.

Projector

Full screen →

Ana - Lab 1

Ben - Lab 2

Outfit Lab

Dress one character together. Every button runs real Git.

Your name

Join

The app puts you in a lab.

OUTFIT LAB

A real class run: 3 labs of 2 students. The teacher presses only Next. Every button runs real Git.

0:00

OUTFIT LAB

A real class run: 3 labs of 2 students. The teacher presses only Next. Every button runs real Git.

Outfit Lab - Teacher6 people - 6 online

Teacher consoleJoin QRStart presenting

SCENE 1 OF 19 - PLAN 000

Join

0:13 / 3:00 - on plan

SAYNo Git lecture today. You'll hit seven problems, and your lab invents a fix for each. Every button runs real Git.

DOWait until the count matches the room. The app balances the labs. Labs lock at Step 1.

Next: Step 0: Everyone, one outfitBack

Lab 12 people

Lab 22 people

Lab 32 people

ON THE PROJECTOR NOW

ADDRESSlocalhost:3310

Open the link and type your name. The app picks your lab.

Lab 1Ana, Raj

Lab 2Mei, Ben

Lab 3Kim, Tom

NEXT

Step 0: Everyone, one outfit

Your lab shares one outfit. Change any part, any time.

Projector

Full screen →

Ana - Lab 1

Ben - Lab 2

Outfit Lab

You're in Lab 1

Your lab dresses one character together. The class starts in a moment.

IN LAB 1

Ana (you)Raj

Labs can still change until Step 1.

HATcap

GLASSESround glasses

TOPT-shirt

SHOESsneakers

Click a part to change it.

Outfit Lab

You're in Lab 2

Your lab dresses one character together. The class starts in a moment.

IN LAB 2

MeiBen (you)

Labs can still change until Step 1.

HATcap

GLASSESround glasses

TOPT-shirt

SHOESsneakers

Click a part to change it.

JOIN

Students open the link and type a name. The app puts them in a lab.

0:04

JOIN

Students open the link and type a name. The app puts them in a lab.

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 2 OF 19 · PLAN 0-03

Step 0 · Everyone, one outfit0:05 / 1:30 · 3 min ahead

SAYYour lab shares one outfit. Change any part, any time. Go.

DOAfter 90 seconds, say "Hands off." Then ask.

ASKWhat did your outfit look like a minute ago? Who changed the shoes?

Show on projectorAnswer

Next: Step 0 · What would fix it?Back

Lab 12 people

Lab 22 people

Lab 32 people

ON THE PROJECTOR NOW

STEP 0 Everyone, one outfitYour lab shares one outfit. Change any part, any time.

1:25

NEXT

Step 0 · What would fix it?

STEP 0

Everyone, one outfit

Your lab shares one outfit. Change any part, any time.

Stuck? Hint

HAT

helmet

change >

GLASSES

round glasses

change >

TOP

safety vest

change >

SHOES

sneakers

change >

STEP 0

Everyone, one outfit

Your lab shares one outfit. Change any part, any time.

Stuck? Hint

HAT

sun hat

change >

GLASSES

round glasses

change >

TOP

T-shirt

change >

SHOES

sandals

change >

STEP 0 · CHAOSEach lab edits one shared outfit, live. Nothing is saved.

no Git yet

0:13

STEP 0 · CHAOS

Each lab edits one shared outfit, live.
Nothing is saved. no Git yet

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 3 OF 19 · PLAN 0-04

Step 0 · What would fix it?0:02 / 3:30 · 4 min ahead

SAYToday you'll explain how Git saves, splits, combines and shares work. You'll predict when Git says no, and pick the right undo.

ASKWhat rule would fix this?

Show on projectorAnswer

2/6 answeredShow answers on projectorwith names

Kim Lab 3 · Save every version, with a name on it.

Raj Lab 1 · Keep a history of who changed what.

Takeaways written: 3/6

BOARD0 · Save every version, with a name on it.

Next: Step 1 · Save every versionBack

Lab 1

Lab 2

ON THE PROJECTOR NOW

STEP 0 Everyone, one outfitWhat rule would fix this?

Nothing was saved. Nobody knows who changed what.

1:25

NEXT

Step 1 · Save every versionChange one part, then press Save card. Everyone saves once.

STEP 0

What would fix it?

QUESTION

What rule would fix this?

Your answer · Enter saves (optional)

MY TAKEAWAY

Without saves, nobody knows w

Saved ✓

HAT

helmet

change >

GLASSES

round glasses

change >

TOP

safety vest

change >

SHOES

sneakers

change >

STEP 0

What would fix it?

QUESTION

What rule would fix this?

Your answer · Enter saves (optional)

MY TAKEAWAY

red draft and no history is chaos.

Saved ✓

HAT

sun hat

change >

GLASSES

round glasses

change >

TOP

T-shirt

change >

SHOES

sandals

change >

STEP 0 · CHAOSNobody can say who changed what. Students answer in the app: what rule would fix it?

0:20

STEP 0 · CHAOS

Nobody can say who changed what.
Students answer in the app: what rule would fix it?

Outfit Lab - Teacher 6 people · 6 online

Teacher console Join QR Start presenting

SCENE 4 OF 19 · PLAN 0:08

Step 1 · Save every version

0:07 / 4:00 · 8 min ahead

SAY From now on, every save makes a card.

DO Ask first. Then say "Go."

ASK What should each saved version hold?

Show on projector Answer

Labs done: 3/3

Next: Step 1 · How Git does it

Back

ON THE PROJECTOR NOW

Save every version

Change one part, then press Save card. Everyone saves once.

Lab 1 Lab 2 Lab 3

Everyone saved a card (2/2)

Everyone saved a card (2/2)

Everyone saved a card (2/2)

coatsneakers

SHOES sneakers change

All saved

Your lab's cards git log

2682fc3 OutfitLab Start

59fc6ba Ana HAT: cap → sunhat

3e22873 Raj TOP: t-shirt → coat

← points to the card before

STEP 1 · SAVE Everyone changes one part and presses Save card. Each save is a commit, with a name and a time. git commit

0:24

STEP 1 · SAVE

Everyone changes one part and presses Save card. Each save is a commit, with a name and a time. `git commit`

Outfit Lab - Teacher 6 people · 6 online

Teacher console Join QR Start presenting

SCENE 4 OF 19 · PLAN 0:08

Step 1 · Save every version

0:14 / 4:00 · 8 min ahead

SAY From now on, every save makes a card.

DO Ask first. Then say "Go."

ASK What should each saved version hold?

Show on projector Answer

Labs done: 3/3

Next: Step 1 · How Git does it

Back

ON THE PROJECTOR NOW

Save every version

Change one part, then press Save card. Everyone saves once.

Lab 1 Lab 2 Lab 3

Everyone saved a card (2/2)

Everyone saved a card (2/2)

Everyone saved a card (2/2)

GLASSES round glasses change

TOP coat change

SHOES sneakers change

All saved

Your lab's cards git log

2682fc3 OutfitLab Start

59fc6ba Ana HAT: cap → sunhat

3e22873 Raj TOP: t-shirt → coat

← points to the card before

STEP 1 · SAVE Click a card: the commit object Git stored. A tree, the parent, and an author and a committer, each with a time. `git cat-file -p`

0:32

STEP 1 · SAVE

Click a card: the commit object Git stored. A tree, the parent, and an author and a committer, each with a time. `git cat-file -p`

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 5 OF 19 · PLAN 0-15

Step 1 · How Git does it0:08 / 3:00 · 11 min ahead

SAYgit commit stores the whole outfit, the card before, your name, the time and a message. The ID is computed from all of it, so any change makes a new card. The name and the time are whatever your laptop says.

ASKWhy do arrows point back, never forward?

0/6 answeredShow answers on projectorwith namesTakeaways written: 2/6

BOARD1. Card (commit): a full snapshot + its parent. Never changes.

Next: Step 2 · Try two ideas at onceBack

ON THE PROJECTOR NOW

Save every version

git commit

WHAT IT ISa saved snapshot of the whole project.

WHAT IT DOESrecords the exact state, who saved it, when, and the commit before it.

HOW GIT DOES ITeach file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

NEXT

Step 2 · Try two ideas at oncePair A builds fancy. Pair B builds sporty. Each on its own sticky note.

STEP 1Save every version

Why do arrows point back, never forward?

git commit

WHAT IT ISa saved snapshot of the whole project.

WHAT IT DOESrecords the exact state, who saved it, when, and the commit before it.

HOW GIT DOES ITeach file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

Outfit LabLab 1 · Ana · Step 1 · How Git does itAna · Lab 1

STEP 1How Git does it

QUESTIONWhy do arrows point back, never forward?

YOUR ANSWER - Enter saves (optional)

MY TAKEAWAYA card is a full snapshot plus its t

Behind the doorwhat Git did

git commit

WHAT IT ISa saved snapshot of the whole project.

WHAT IT DOESrecords the exact state, who saved it, when, and the commit before it.

HOW GIT DOES ITeach file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

Your lab's cardsgit log

Outfit LabLab 2 · Ben · Step 1 · How Git does itBen · Lab 2

STEP 1How Git does it

QUESTIONWhy do arrows point back, never forward?

YOUR ANSWER - Enter saves (optional)

MY TAKEAWAYlaptop for my name and the time.

Behind the doorwhat Git did

git commit

WHAT IT ISa saved snapshot of the whole project.

WHAT IT DOESrecords the exact state, who saved it, when, and the commit before it.

HOW GIT DOES ITeach file is stored as a blob, the folder as a tree. A commit object = tree ID + parent ID(s) + author + committer + message. Its ID is the SHA-1 of that object, so any change gives a new ID. Nothing is edited in place.

Git stores the name and clock your laptop gives it. It checks neither.

Your lab's cardsgit log

STEP 1 · SAVEOne technical card per tool. Git stores the name and clock your laptop gives it. It checks neither.

git commit

0:41

STEP 1 · SAVE

One technical card per tool. Git stores the name and clock your laptop gives it. It checks neither. `git commit`

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 6 OF 19 · PLAN 0-15

Step 2 · Try two ideas at once0:15 / 4:00 · 14 min ahead

SAYThe client wants two ideas tried at once. In one draft, you overwrite each other, like Step 0.

DOAsk first. Both missions change TOP on purpose. Don't tell them.

ASKCards never change. How does each half of your lab find its own newest card?

Labs done: 3/3

Next: Step 2 · How Git does itBack

ON THE PROJECTOR NOW

Try two ideas at once

Pair A builds fancy. Pair B builds sporty. Each on its own sticky note.

Lab 1Lab 2Lab 3

Step 2 · How Git does it

NEXT

Step 2 · How Git does it

STEP 2Try two ideas at once

Pair A builds fancy. Pair B builds sporty. Each on its own sticky note.

Lab 1Lab 2Lab 3

Outfit LabLab 12 people

HAT → TOP →

Save card.

You're in Pair A, on your own (change)

fancy has

sporty has

Done ✓. Wait for the class.

Bonus · Switch to main and back. Watch the outfit change. Look, don't edit.

Behind the doorwhat Git did

Your lab's cardsgit log

2682fc359f6c8ba3e22873c718684e443357

← points to the card before sticky note your pin (HEAD): the note you're on

Outfit LabLab 22 people

TOP → SHOES →

Save card.

You're in Pair B, on your own (change)

fancy has

sporty has

Done ✓. Wait for the class.

Bonus · Switch to main and back. Watch the outfit change. Look, don't edit.

Behind the doorwhat Git did

Your lab's cardsgit log

2682fc3020568bcfc9344d173bc88e9f56bb

← points to the card before sticky note your pin (HEAD): the note you're on

STEP 2 · TWO IDEASPair A builds fancy and Pair B builds sporty, each on its own sticky note. Both change TOP.

git switch -c

0:50

STEP 2 · TWO IDEAS

Pair A builds fancy and Pair B builds sporty, each on its own sticky note. Both change TOP. `git switch -c`

Outfit Lab · Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 7 OF 19 · PLAN 0-190:02 / 3:00 · 18 min ahead

STEP 2 · How Git does it

SAYA branch is a tiny file holding one card's ID. Your pin (HEAD) says which note you're on. Saving moves only that note.

DOIf asked: here each sticky note keeps its own shared draft, a teaching choice. In real Git, branches share one working tree (unless you add one with git worktrees).

ASKWhere is the original outfit now? Did anything get copied?

0/6 answeredShow answers on projectorwith names

Takeaways written: 3/6

BOARD2. Sticky note (branch): a label on one card. Saving moves it.

Next: Step 3 · Make one outfit from bothBack

ON THE PROJECTOR NOW

Try two ideas at onceWhere is the original outfit now? Did anything get copied?

git branch

WHAT IT ISa movable label on one commit.

WHAT IT DOESlets you try an idea without touching main.

HOW GIT DOES ITa branch is a tiny file (.git/refs/heads/<name>) holding one commit ID. Committing moves it forward. HEAD records which branch you are on. Creating a branch copies nothing.

NEXT

Step 3 · Make one outfit from bothOn main: merge fancy, delete the fancy note, then merge sporty.

STEP 2

How Git does it

QUESTIONWhere is the original outfit now? Did anything get copied?

Your answer: Enter saves (optional)

MY TAKEAWAYA branch is a label on one card.

Saved ✓

Behind the doorwhat Git did

git branch

WHAT IT ISa movable label on one commit.

WHAT IT DOESlets you try an idea without touching main.

HOW GIT DOES ITa branch is a tiny file (.git/refs/heads/<name>) holding one commit ID. Committing moves it forward. HEAD records which branch you are on. Creating a branch copies nothing.

Your lab's cardsgit log

STEP 2

How Git does it

QUESTIONWhere is the original outfit now? Did anything get copied?

Your answer: Enter saves (optional)

MY TAKEAWAYwing moves only the note I'm on.

Saved ✓

Behind the doorwhat Git did

git branch

WHAT IT ISa movable label on one commit.

WHAT IT DOESlets you try an idea without touching main.

HOW GIT DOES ITa branch is a tiny file (.git/refs/heads/<name>) holding one commit ID. Committing moves it forward. HEAD records which branch you are on. Creating a branch copies nothing.

Your lab's cardsgit log

STEP 2 · TWO IDEASA branch is a tiny file that holds one commit ID. Making one copies nothing.git branch

1:07

STEP 2 · TWO IDEAS

A branch is a tiny file that holds one commit ID. Making one copies nothing. git branch

Outfit Lab · Lab 1 · Ana · Step 3 · Make one outfit from both

Ana · Lab 1?

STEP 3

Make one outfit from both

One person presses, everyone watches. Swap each step.

The client wants one outfit with both ideas. On main: merge fancy. Then merge sporty.

☒ main has fancy

☐ The fancy note is deleted

☐ main has sporty

Stuck? Hint

Behind the door

Shared draft

You're on: main

HATsun hat

GLASSESround glasses

TOPcoat

SHOESsneakers

In this step, main changes only by merging. Press Merge.

Your lab's cardsgit log

YOUR MISSION

Merge fancy into main

git merge

Save cardgit commit

New sticky notegit switch -c

Switch to fancygit switch

mainYOUsporty

You're on main now.

STEP 3 · COMBINEOne person per lab merges both ideas into main.git merge

1:12

STEP 3 · COMBINE

One person per lab merges both ideas into main. git merge

Outfit Lab • Lab 1 • Ana • Step 3 • Make one outfit from both

fancy. Then merge sporty.

YOUR MISSION
Delete the fancy note
(`git branch -d fancy`).

- ☒ main has fancy
- ☐ The fancy note is deleted
- ☐ main has sporty

Stuck? Hint

Behind the door
what Git did

In this step, main changes only by merging.
Press Merge.

Save card
`git commit`

New sticky note
`git switch -c`

Switch to
sporty
`git switch`

Your lab's cards `git log`

main slid forward to ee43357. No new card.

STEP 3 • COMBINE Merge fancy into main: main only slides forward. A fast-forward makes no new card.

`git merge`

1:15

STEP 3 • COMBINE

Merge fancy into main: main only slides forward. A fast-forward makes no new card.

`git merge`

Outfit Lab • Lab 1 • Ana • Step 3 • Make one outfit from both

fancy. Then merge sporty.

- ☒ main has fancy
- ☒ The fancy note is deleted
- ☐ main has sporty

Stuck? Hint

Behind the door
what Git did

In this step, main changes only by merging.
Press Merge.

Switch to
sporty
`git switch`

Your lab's cards `git log`

Deleted the fancy note. Its cards stay.

STEP 3 • COMBINE Delete the fancy note. Git allows it because main already has its cards.

`git branch -d fancy`

1:20

STEP 3 • COMBINE

Delete the fancy note. Git allows it because main already has its cards. `git branch -d fancy`

Outfit Lab • Lab 1 • Ana - Lab 1

STEP 3

Make one outfit from both

One person presses watches. Swap each other's watches. Swap each other's watches. Swap each other's watches.

The client wants a fancy outfit. The client wants a fancy outfit. The client wants a fancy outfit.

main has fancy

The fancy note

main has sporty

Stuck? Hint

Where you split merge base

main

sporty

HAT ✓ top hat

GLASSES ✓ round glasses

TOP

Keep main's shirt & tie

Keep sporty's jersey

Pick another...

SHOES ✓ boots

Finish merge

git add outfit.txt

git commit

Cancel merge git merge --abort

Behind the door • the file Git wrote

STEP 3 • COMBINE Merge sporty: HAT and SHOES combine by themselves. TOP changed on both sides, so a person picks. git merge

1:26

STEP 3 • COMBINE

Merge sporty: HAT and SHOES combine by themselves. TOP changed on both sides, so a person picks. git merge

Outfit Lab • Lab 1 • Ana • Step 3 • Make one outfit from both

main has sporty

Done ✓. Wait for the class.

Bonus • Open the merge card. Why two parents?

Behind the door what Git did

Switch to sporty git switch

Your lab's cards git log

← points to the card before

sticky note

YOU your pin (HEAD): the note you're on

Saved merge card 3e3272a. It has two parents.

STEP 3 • COMBINE Finish merge writes a merge card with two parents. git commit

1:35

STEP 3 • COMBINE

Finish merge writes a merge card with two parents. git commit

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 9 OF 19 · PLAN 0:09

Step 3 · How Git does it0:03 / 4:00 · 27 min ahead

SAYThe first merge only slid the note: a fast-forward, no new card. Then Git compared each side with the card where you split. Only TOP changed on both sides, so only TOP needed you.

DOIf asked: main's own safety diary still says "merge fancy: Fast-forward". It is local, it expires, and it never reaches the Wall.

ASKWhich cards were made on fancy?

Show on projectorAnswer

2/6 answeredShow answers on projectorwith names

Ana Lab 1 · No way to tell: A card does not record its branch.

Tom Lab 3 · A commit stores its parents, not its branch.

Takeaways written: 3/6

BOARD3 · Merge: compare both sides with the card they share.

ON THE PROJECTOR NOW

Make one outfit from bothWhich cards were made on fancy?

git merge

What it iscombining two lines of work.

What it doeskeeps a change made on one side. It stops for a person when both sides changed the same lines.

How Git does itGit finds the merge base (the last common commit) and compares each side with it (a 3-way merge). Then it writes a merge commit with two parents. If one side is ahead, it fast-forwards: only the label moves, and no merge commit is made.

NextBreakStart to the 4-minute break. Labs can still finish Step 3.

Outfit LabAna · Lab 1 · Step 3 · How Git does it

Ana · Lab 1

STEP 3How Git does it

QUESTIONWhich cards were made on fancy?

No way to tell. A card does not record its branch.

MY TAKEAWAYMerge compares both sides with

Behind the doorwhat Git did

git merge

WHAT IT IScombining two lines of work.

WHAT IT DOESkeeps a change made on one side. It stops for a person when both sides changed the same lines.

HOW GIT DOES ITGit finds the merge base (the last common commit) and compares each side with it (a 3-way merge). Then it writes a merge commit with two parents. If one side is ahead, it fast-forwards: only the label moves, and no merge commit is made.

Your lab's cardsgit log

STEP 3 · COMBINEWhich cards were made on fancy? No way to tell: a commit does not record its branch.

git merge

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 10 OF 19 · PLAN 0:33

Break0:03 / 4:00 · 31 min ahead

SAYBreak. Back in 4 minutes.

DOWrite the return time on the board. Help any lab that is not done.

Restart the break timer

Labs done: 2/3

Next: Step 4 · Put your outfit on the WallBack

Lab 12 people

Lab 22 people

Lab 32 people

ON THE PROJECTOR NOW

BREAK

3:57

Back at 8:38 AM

NextStep 4 · Put your outfit on the WallMake your change, Save card, Send to Wall. The first lab there wins. Sends Lab 1's outfit to the Wall. Every lab becomes a copy of it. Back does not undo this. Whose outfit goes to the WallLab 1

Outfit LabAna · Lab 1 · Break

Ana · Lab 1

Break · back at 8:38 AM

STEP 3Make one outfit from both

One person presses, everyone watches. Swap each step.

The client wants one outfit with both ideas. On main: merge fancy. Then merge sporty.

main has fancy

The fancy note is deleted

main has sporty

Done ✓. Wait for the class.

Shared draft

You're on: main

HATtop hat

GLASSESround glasses

TOPshirt & tie

SHOESboots

In this step, main changes only by merging. Press Merge.

Merge sporty into main

Delete sticky note sporty

Save cardgit commit

New sticky notegit switch -c

Switch to sporty

Your lab's cardsgit log

BREAKA 4-minute break. A lab that is behind can still finish Step 3.

1:45

STEP 3 · COMBINE

Which cards were made on fancy? No way to tell: a commit does not record its branch.

git merge

1:50

BREAK

A 4-minute break. A lab that is behind can still finish Step 3.

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

7:56

SCENE 11 OF 16 · PLAN 0:37

Step 4 · Put your outfit on the Wall0:04 / 8:00 · 35 min ahead

SAYThe Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is git clone). It starts as Lab 1's outfit. Put your outfit on the Wall. The first lab there wins.

DODon't task first: students hit the refusal themselves. Each refused lab gets its way in the app: Combine (merge) or Replay on top (rebase).

Labs done: 0/3

Next: Step 4 · How Git does it

Back

Lab 12 peopleLab 22 peopleLab 32 people

ON THE PROJECTOR NOW

STEP 4 Put your outfit on the Wall7:56

Make your change, Save card, Send to Wall. The first lab there wins.

THE WALL

Lab 1Lab 2Lab 3

Step 4 - How Git does it

Projector

Outfit Lab · Lab 1 · Ana · Step 4 · Put your outfit on the Wall

Ana · Lab 1

STEP 4 Put your outfit on the Wall

One person presses, everyone watches. Swap each step.

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is git clone). It starts as your lab's outfit, so your lab's cards are now a copy of it. Make your lab's change, **Save card**, then **Send to Wall**.

YOUR MISSIONHAT → 🧢. Change nothing else.

Shared draft

You're on: main

HAT top hat change >

GLASSES round glasses change >

TOP shirt & tie change >

SHOES boots change >

All saved

Send to Wallgit push

Get the Wall's cards: Combine (merge) or Replay on top (rebase)

NEWGet & combinegit pull --no-rebase

NEWReplay on topgit pull --rebase

Save cardgit commit

New sticky notegit switch -c

STEP 4 Put your outfit on the Wall

One person presses, everyone watches. Swap each step.

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is git clone). It starts as Lab 1's outfit, so your lab's cards are now a copy of it. Make your lab's change, **Save card**, then **Send to Wall**.

YOUR MISSIONSHOES → 👢. Change nothing else.

Shared draft

You're on: main

HAT top hat change >

GLASSES round glasses change >

TOP shirt & tie change >

SHOES boots change >

All saved

Send to Wallgit push

Get the Wall's cards: Combine (merge) or Replay on top (rebase)

NEWGet & combinegit pull --no-rebase

NEWReplay on topgit pull --rebase

Save cardgit commit

New sticky notegit switch -c

STEP 4 · SHARE

Next puts Lab 1's outfit on the Wall, the class's shared copy. Every lab is now a full copy of it.

git clone

1:55

STEP 4 · SHARE

Next puts Lab 1's outfit on the Wall, the class's shared copy. Every lab is now a full copy of it. `git clone`

Outfit Lab · Lab 1 · Ana · Step 4 · Put your outfit on the Wall

Ana · Lab 1

STEP 4 Put your outfit on the Wall

One person presses, everyone watches. Swap each step.

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is git clone). It starts as your lab's outfit, so your lab's cards are now a copy of it. Make your lab's change, **Save card**, then **Send to Wall**.

YOUR MISSIONHAT → 🧢. Change nothing else.

Shared draft

You're on: main

HAT crown change >

GLASSES round glasses change >

TOP shirt & tie change >

SHOES boots change >

All saved

Send to Wallgit push

Get the Wall's cards: Combine (merge) or Replay on top (rebase)

NEWGet & combinegit pull --no-rebase

NEWReplay on topgit pull --rebase

Save cardgit commit

git switch -c

STEP 4 · SHARE

Each lab makes one change and saves. Lab 3 sends first: the Wall moves forward.

git push

1:59

STEP 4 · SHARE

Each lab makes one change and saves. Lab 3 sends first: the Wall moves forward. `git push`

Outfit Lab • Lab 1 • Ana • Step 4 • Put your outfit on the Wall

STEP 4

Put your outfit on the Wall

One person presses, everyone watches. Swap each step.

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is `git clone`). It starts as your lab's outfit, so your lab's cards are now a copy of it. Make your lab's change, **Save card**, then **Send to Wall**.

YOUR MISSION
HAT → 👑. Change nothing else. The Wall refused your

Shared draft You're on: **main**

Item	Current	Action
HAT	crown	change >
GLASSES	round glasses	change >
TOP	shirt & tie	change >
SHOES	boots	change >

All saved

Send to Wall NEW
`git push`

Get the Wall's cards: **Combine (merge)** or **Replay on top (rebase)**

YOUR MISSION
Get & combine
`git pull --no-rebase`

Replay on top
`git pull --rebase`

Refused: the Wall has cards your main doesn't. Use Combine (merge): press Get & combine, then send again.

STEP 4 • SHARE Ana's send is refused: the Wall has a card her main lacks. A push may only move the Wall forward. `git push`

2:05

STEP 4 • SHARE

Ana's send is refused: the Wall has a card her main lacks. A push may only move the Wall forward. `git push`

Outfit Lab • Lab 1 • Ana • Step 4 • Put your outfit on the Wall

Done ✓. Wait for the class.

Bonus • Open your lab's card on the Wall. Is its ID the one you saved?

Behind the door
what Git did

main YOU

2682fc3 ← 2 older cards ee43357 3e3272a 29a40a0 6a752bd

c7186b4 cbc51ec

← points to the card before sticky note YOU your pin (HEAD): the note you're on the Wall, last time you checked

• **The Wall** the class's shared copy

Sent! The Wall moved to 6a752bd.

STEP 4 • SHARE Lab 1's way is Combine: fetch the Wall's cards, write a merge card with two parents, then send. `git pull --no-rebase`

2:08

STEP 4 • SHARE

Lab 1's way is Combine: fetch the Wall's cards, write a merge card with two parents, then send. `git pull --no-rebase`

Outfit Lab • Lab 2 • Ben • Step 4 • Put your outfit on the Wall

Ben - Lab 2

STEP 4

Put your outfit on the Wall

One person presses, everyone watches. Swap each step.

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (that is `git clone`). It starts as Lab 1's outfit, so your lab's cards are now a copy of it. Make your lab's change, **Save card**, then **Send to Wall**.

YOUR MISSION
SHOES → 🚗. Change nothing else. The Wall refused your

Shared draft You're on: **main**

Item	Change
HAT top hat	change >
GLASSES round glasses	change >
TOP shirt & tie	change >
SHOES skates	change >

All saved

Send to Wall NEW
`git push`

Get the Wall's cards: **Combine (merge)** or **Replay on top (rebase)**

Get & combine
`git pull --no-rebase`

YOUR MISSION
Replay on top
`git pull --rebase`

Refused: the Wall has cards your main doesn't. Use Replay on top (rebase): press **Replay on top**, then send again.

STEP 4 • SHARE Ben's send is refused too. Lab 2's way is Replay on top. `git push`

2:17

STEP 4 • SHARE

Ben's send is refused too. Lab 2's way is Replay on top. `git push`

Outfit Lab • Lab 2 • Ben • Step 4 • Put your outfit on the Wall

Ben - Lab 2

☒ Your change is on the Wall
☐ Your cards match the Wall

Stuck? Hint

Behind the door
what Git did

← points to the card before sticky note **YOU** your pin (HEAD): the note you're on
the Wall, last time you checked

The Wall the class's shared copy

STEP 4 • SHARE Replay on top: Git writes a new card with the same change, on the Wall's newest card. New parent, new ID. `git pull --rebase`

2:20

STEP 4 • SHARE

Replay on top: Git writes a new card with the same change, on the Wall's newest card. New parent, new ID. `git pull --rebase`

Outfit Lab • Lab 2 • Ben • Step 4 • Put your outfit on the Wall

The Wall is the class's shared copy, like GitHub. Your lab already has a full copy of it (it is `git clone`). It starts as Lab 1's outfit, so your lab's cards are now a copy of it. Make your change, **Save card**, then **Send to Wall**.

YOUR MISSION
SHOES → 🦶. Change nothing else. The Wall refused your send. Use **Replay on top (rebase)**: press **Replay on top**, then **Send to Wall** again.

☐ Your change is on the Wall
☐ Your cards match the Wall

STEP 4 • SHARE Same author and author time, a new committer time. The original card is only in the safety diary. `git rebase`

CARD ID
0b68b0b
0b68b0b80fa425f0bfa1c044b8cc2bc6aaabbd11

SHOES: boots → skates

Author Ben at 8:34:33 · committed at 8:34:51

Parent 6a752bd

Replay of a9352aa only in your safety diary (reflog)

Show what Git stored

Get the Wall's cards: **Combine (merge)** or **Replay on top (rebase)**

1 --no-rebase

MISSION

on top

--rebase

New sticky note

git switch -c

2682fc3 3e3272a 29a40a0 6a752bd 0b68b0b

← 3 older cards

wall/main main YOU

2:26

STEP 4 • SHARE

Same author and author time, a new committer time. The original card is only in the safety diary. `git rebase`

STEP 4 Put your outfit on the Wall

The replayed card has the same change, author and author time. Why does it have a new ID?

git push / git pull **git rebase**

WHAT IT IS	WHAT IT DOES	WHAT IT IS	WHAT IT DOES
sharing commits with a copy elsewhere (the Wall).	push sends your new commits and moves the remote branch. pull brings theirs in.	replaying your commits on top of another branch.	gives a straight history with no merge commit.

HOW GIT DOES IT

copies exchange only the objects the other side lacks; IDs come from content, so copies agree without coordination. A push is accepted only if it is a fast-forward of the remote branch. Otherwise fetch first, then merge or rebase.

for each of your commits, Git applies its change to the new base and writes a new commit. The new parent gives it a new ID. Author and author date are kept. The originals become unreachable and stay in your reflog for a while.

THE WALL • LAB 1'S PATH TO MAIN (BOLD)

2682fc3 3e3272a 29a40a0 6a752bd 0b68b0b

← 3 older cards

c7186b4 cbc51ec

main

- Lab 3 🧑 · made 8:34 → on the Wall 8:34 · 2 s
- Lab 1 🧑 · made 8:34 → on the Wall 8:34 · 7 s
The path ends at its own merge card, with two parents.
- Lab 2 🧑 · made 8:34 → on the Wall 8:35 · 31 s
Its original card is not on the Wall. The path starts at a copy.

STEP 4 • SHARE On the Wall: Lab 1's merge card has two parents. Lab 2's copy is a straight line with a new ID. `git push · git rebase`

Projector

2:38

STEP 4 • SHARE

On the Wall: Lab 1's merge card has two parents. Lab 2's copy is a straight line with a new ID. `git push · git rebase`

Outfit Lab · Teacher 6 people · 6 online

Teacher console

Join QR Start presenting

Show on projector ▶ Answer

0/6 answered Show answers on projector ☐ with names

Takeaways written: 0/6

BOARD 4. The Wall only moves forward. Behind? Combine (merge), or replay on top (rebase: new IDs).

PATHS Lab 3 🧐 made 8:34 → on the Wall 8:34 · 2 s
Lab 1 🧐 made 8:34 → on the Wall 8:34 · 7 s
Lab 2 🧐 made 8:34 → on the Wall 8:35 · 31 s · starts at a copy

Sabotage: the Intern's 🧐 card

Next: Step 5 · Oops: undo a shared mistake Back

ON THE PROJECTOR NOW

STEP 4 · Put your outfit on the Wall

The replayed card has the same change, author and author time. Why does it have a new ID?

git push / git pull git rebase

git push / git pull: pushing commits with a copy elsewhere (the Wall). For each of your commits, git applies the change to the new base and writes a new commit. The new commit's parent is the commit you pushed. The original commit and any in-progress changes are detached and only in your working space.

git rebase: rebasing your new commits and moves the entire branch's pointer forward. For each of your commits, git applies the change to the new base and writes a new commit. The new commit's parent is the commit you pushed. The original commit and any in-progress changes are detached and only in your working space.

THE WALL · LAB 3 PATH TO MAIN (BOLD)

Lab 3 → made 8:34 → on the Wall 8:34 · 2 s
Lab 1 → made 8:34 → on the Wall 8:34 · 7 s
Lab 2 → made 8:34 → on the Wall 8:35 · 31 s

NEXT

Step 5 · Oops: undo a shared mistake

A 🧐 card reached the Wall. Remove it without breaking anyone's copy.

Puts the Intern's 🧐 card on the Wall, if you haven't yet. Each lab with nothing unsent gets it (a fast-forward).

STEP 4 · SHARE The teacher sees each change's path to main, with this class's real times: made → on the Wall. code velocity

2:46

STEP 4 · SHARE

The teacher sees each change's path to main, with this class's real times: made → on the Wall. code velocity

Outfit Lab Lab 2 · Ben · Step 4 · How Git does it

Ben · Lab 2

Each change's path to main (integration path). Click one to mark it.

2682fc3 ← 3 older cards

3e3272a

29a40a0

6a752bd

0b68b0b main

c7186b4

cbc51ec

copy · new ID

merge · 2 parents

Lab 3 · GLASSES → 🧐 made 8:34 → on the Wall 8:34 · 2 s

Lab 1 · HAT → 🧐 made 8:34 → on the Wall 8:34 · 7 s
The path ends at its own merge card, with two parents.

Lab 2 · SHOES → 🧐 made 8:34 → on the Wall 8:35 · 31 s
Its original card is not on the Wall. The path starts at a copy.

STEP 4 · SHARE Ben's lab: its original card is not on the Wall. The path starts at a copy. git rebase

2:53

STEP 4 · SHARE

Ben's lab: its original card is not on the Wall. The path starts at a copy. git rebase

Outfit Lab - Teacher 6 people · 6 online

Teacher console Join QR Start presenting

SCENE 13 OF 19 · PLAN 5:50 0:04 / 6:00 · 47 min ahead

Step 5 - Oops: undo a shared mistake

DO The Intern pushed a "tiny style fix" to the Wall. Your lab has it too.

ASK How do you get rid of it?

Show on projector Answer

Sabotage: the Intern's 🤖 card

Labs done: 0/3

Next: Step 5 - How Git does it

ON THE PROJECTOR NOW

STEP 5 - Oops: undo a shared mistake 5:56

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

THE WALL

2023f13 Outfit Lab Start

23a0a6 Ana Switch signed + commit

6a712d6 Ana Merge with main

81a63d6 Ben Switch to main

8887ab8 Outfit Lab Merge to main

Lab 1

You undid the 🤖 card yourselves.

No 🤖 on the Wall or in your lab (Send to Wall)

Lab 2

You undid the 🤖 card yourselves.

No 🤖 on the Wall or in your lab (Send to Wall)

Lab 3

You undid the 🤖 card yourselves.

No 🤖 on the Wall or in your lab (Send to Wall)

Projector

Outfit Lab Lab 1 - Ana - Step 5 - Oops: undo a shared mistake

Ana - Lab 1

Outfit Lab Lab 2 - Ben - Step 5 - Oops: undo a shared mistake

Ben - Lab 2

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

YOUR MISSION

Your lab has it now. In your lab's cards, click the 🤖 card. Press **Undo this card**, then **Send to Wall**. Refused? **Get & combine**, then send again.

Shared draft

You're on: **main**

HAT crown change >

GLASSES disguise glasses change >

TOP shirt & tie change >

SHOES skates change >

All saved

Your lab's cards git log

Safety diary git reflog

New: click a card to undo it or move your note back.

Save card git commit

New sticky note git switch -c

Send to Wall git push

Get & combine git pull --rebase

Replay on top git pull --rebase

STEP 5 - UNDO

Next: the Intern pushes a 🤖 card to the Wall. Every lab gets it, as a fast-forward.

git pull

3:04

STEP 5 · UNDO

Next: the Intern pushes a 🤖 card to the Wall. Every lab gets it, as a fast-forward.

git pull

Outfit Lab Lab 2 · Ben · Step 5 - Oops: undo a shared mistake

Ben - Lab 2

?

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

YOUR MISSION

Your lab has it now. Click the card right before 🤖. Press **Move my note back here**, then **Send to Wall**. What happens?

Shared draft

You're on: **main**

HAT crown change >

GLASSES shades change >

TOP shirt & tie change >

SHOES skates change >

All saved

Your lab's cards git log

Safety diary git reflog

New: click a card to undo it or move your note back.

Save card git commit

New sticky note git switch -c

Get & combine

Replay on top

YOUR MISSION

Send to Wall git push

Safety diary git reflog

main moved back to 0b68b0b. Now press **Send to Wall**.

STEP 5 - UNDO

Lab 2 tries Move my note back: main points again at the card before 🤖.

git reset --hard

3:09

STEP 5 · UNDO

Lab 2 tries Move my note back: main points again at the card before 🤖.

git reset --hard

Outfit Lab • Lab 2 • Ben • Step 5 • Oops: undo a shared mistake

Ben • Lab 2

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

YOUR MISSION

Refused. Open the **Safety diary**, then press **Get & combine**. 🤖 back? Click it, press **Undo this card**, then send.

You undid the 🤖 card

Shared draft

You're on: **main**

HAT crown	change ›
GLASSES shades	change ›
TOP shirt & tie	change ›
SHOES skates	change ›

All saved

YOUR MISSION

Safety diary
git reflow

New: click a card to undo it or move your note back.

Save card
git commit

New sticky note
git switch -c

Send to Wall
git push

Get & combine
git pull --no-rebase

Refused: the Wall still has the 🤖 card.
• Open the Safety diary, then press Get & combine.

Your lab's cards

git log

STEP 5 • UNDO The Wall refuses: it still has the 🤖 card, and a push may only move it forward.

git push

3:15

STEP 5 • UNDO

The Wall refuses: it still has the 🤖 card, and a push may only move it forward. `git push`

Outfit Lab • Lab 2 • Ben • Step 5 • Oops: undo a shared mistake

Ben • Lab 2

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🤖 card reached the Wall. Remove it without breaking anyone's copy.

YOUR MISSION

Refused. Open the **Safety diary**, then press **Get & combine**. 🤖 back? Click it, press **Undo this card**, then send.

You undid the 🤖 card

Safety diary

git reflow main

Every place main has been. Newest first. Moving a note deletes nothing. Click a line to open that card.

8:35 AM	🤖	Moved back • 0b68b0b reset: moving to 0b68b0b
8:35 AM	🤖	Got the Wall's cards • 8087cab merge wall/main: Fast-forward
8:34 AM	🤖	Replayed on top • 0b68b0b pull --rebase (finish): refs/heads/main onto 6a75...
8:34 AM	🤖	Saved a card • a9352aa commit: SHOES: boots → skates

Your lab's cards

git log

STEP 5 • UNDO The safety diary lists every place main has been.

git reflow main

3:19

STEP 5 • UNDO

The safety diary lists every place main has been. `git reflow main`

Outfit Lab • Lab 2 • Ben • Step 5 • Oops: undo a shared mistake

Ben · Lab 2

No 🤖 on the Wall or in your lab (Send to Wall)

Stuck? Hint

Behind the door
what Git did

← 5 older cards

2682fc3

29a40a0

6a752bd

0b68b0b

8087cab

cbc51ec

a9352aa

← points to the card before yellow sticky note red pin (YOU) your pin (HEAD): the note you're on

the Wall, last time you checked

• The Wall the class's shared copy

STEP 5 · UNDO Get & combine brings the 🤖 card back: the Wall still had it.

git pull

3:24

STEP 5 · UNDO

Get & combine brings the 🤖 card back: the Wall still had it. `git pull`

Outfit Lab • Lab 2 • Ben • Step 5 • Oops: undo a shared mistake

Ben · Lab 2

No 🤖 on the Wall or in your lab (Send to Wall)

Stuck? Hint

Behind the door
what Git did

← 5 older cards

2682fc3

29a40a0

6a752bd

0b68b0b

8087cab

ccd75c9

cbc51ec

a9352aa

← points to the card before yellow sticky note red pin (YOU) your pin (HEAD): the note you're on

the Wall, last time you checked

• The Wall the class's shared copy

Added fix card ccd75c9. Now press Send to Wall.

STEP 5 · UNDO Undo this card adds a fix card on top. History only grows, so it sends like any card.

git revert

3:29

STEP 5 · UNDO

Undo this card adds a fix card on top. History only grows, so it sends like any card. `git revert`

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 13 OF 19 · PLAN 0:00

Step 5 - Oops: undo a shared mistake0:39 / 6:00 · 47 min ahead

SAYThe intern pushed a "tiny style fix" to the Wall. Your lab has it too.

DOAsk first. Sort the answers into "go back to before it" and "add a card that removes it". Say: "We'll try both."

ASKHow do you get rid of it?

Show on projectorAnswer

Sabotage: the intern's 🐛 card

Labs done: 3/3

Next: Step 5 - How Git does itBack

ON THE PROJECTOR NOW

Step 5 - Oops: undo a shared mistake5:21

🐛 card reached the Wall. Remove it without breaking anyone's copy.

THE WALL

STARTER → Lab 1

STARTER → Lab 2

STARTER → Lab 3

Lab 1

Lab 2

Lab 3

Projector

OUTFIT LAB · Lab 1 · Ana · Step 5 - Oops: undo a shared mistakeAna · Lab 1

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🐛 card reached the Wall. Remove it without breaking anyone's copy.

You undid the 🐛 card yourselves

No 🐛 on the Wall or in your lab (Send to Wall)

Done ✓. Wait for the class.

Bonus - Open the Safety diary.

Shared draft

HAT crown

GLASSES shades

TOP shirt & tie

SHOES skates

All saved

Your lab's cardsgit log

Safety diarygit seflog

New: click a card to undo it or move your note back.

Save cardgit commit

New sticky notegit switch -c

Send to Wallgit push --no-release

Get & combinegit pull --no-release

Replay on topgit pull --rebase

Sent! The Wall moved to 73dc56b.

OUTFIT LAB · Lab 2 · Ben · Step 5 - Oops: undo a shared mistakeBen · Lab 2

STEP 5

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A 🐛 card reached the Wall. Remove it without breaking anyone's copy.

You undid the 🐛 card yourselves

No 🐛 on the Wall or in your lab (Send to Wall)

Done ✓. Wait for the class.

Bonus - Open the Safety diary.

Shared draft

HAT crown

GLASSES shades

TOP shirt & tie

SHOES skates

All saved

Your lab's cardsgit log

Safety diarygit seflog

New: click a card to undo it or move your note back.

Save cardgit commit

New sticky notegit switch -c

Send to Wallgit push --no-release

Get & combinegit pull --no-release

Replay on topgit pull --rebase

STEP 5 · UNDOAna undoes it too. Her send is refused, so she combines the two fix cards (no conflict) and sends.git revert

3:36

STEP 5 · UNDO

Ana undoes it too. Her send is refused, so she combines the two fix cards (no conflict) and sends. `git revert`

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 14 OF 19 · PLAN 0:00

Step 5 - How Git does it0:03 / 3:00 · 52 min ahead

SAYgit revert adds a card that undoes the old one, so it sends like any card. git reset moves your note back, but the Wall still has the card.

ASKWhy is adding a fix card safe, but moving back is not?

Show on projectorAnswer

0/6 answeredShow answers on projectorwith names

Takeaways written: 3/6

5. Shared mistake: add a fix card (revert). Move back (reset) only if nobody has the card.

Next: Step 6 - The boss wants it cleanBack

ON THE PROJECTOR NOW

Step 5 - How Git does it

git revert / git reset

WHAT IT IS

WHAT IT DOES

HOW GIT DOES IT

Step 6 - The boss wants it clean

Lab 1 replaces the Wall with one clean card. Everyone else: watch.

Asks the Wall who added the 🐛 boots. Only Lab 1 can replace the Wall.

Boss lab: Lab 1

OUTFIT LAB · Lab 1 · Ana · Step 5 - How Git does itAna · Lab 1

STEP 5

How Git does it

QUESTION

Why is adding a fix card safe, but moving back is not?

Your answer - Enter saves (optional)

MY TAKEAWAY

Shared mistake: revert, don't res

Behind the door

what Git did

git revert / git reset

WHAT IT IS

WHAT IT DOES

HOW GIT DOES IT

Your lab's cardsgit log

OUTFIT LAB · Lab 2 · Ben · Step 5 - How Git does itBen · Lab 2

STEP 5

How Git does it

QUESTION

Why is adding a fix card safe, but moving back is not?

Your answer - Enter saves (optional)

MY TAKEAWAY

I remembers where my note was.

Behind the door

what Git did

git revert / git reset

WHAT IT IS

WHAT IT DOES

HOW GIT DOES IT

Your lab's cardsgit log

STEP 5 · UNDOrevert is safe on shared history. reset only if nobody else has those cards.git revert · git reset

3:46

STEP 5 · UNDO

revert is safe on shared history. reset only if nobody else has those cards. `git revert` · `git reset`

STEP 6**The boss wants it clean**

3:54

Lab 1 replaces the Wall with one clean card. Everyone else: watch.

THE WALL



BEFORE THE CLEAN-UP · WHO FIRST ADDED 🦶?

Wall: Raj (Lab 1), c7186b4

Lab 1: Raj (Lab 1), c7186b4

Lab 2: Raj (Lab 1), c7186b4

Lab 3: Raj (Lab 1), c7186b4

Lab 1

☐ The Wall has one clean card

Lab 2

☐ The Wall has one clean card

Lab 3

☐ The Wall has one clean card

Projector

STEP 6 · CLEAN UP Next asks the Wall who first added the 🦶 boots. It answers: Raj (Lab 1), card c7186b4.`git log`

3:51

STEP 6 · CLEAN UP

Next asks the Wall who first added the 🦶 boots. It answers: Raj (Lab 1), card c7186b4.

`git log`

3:57

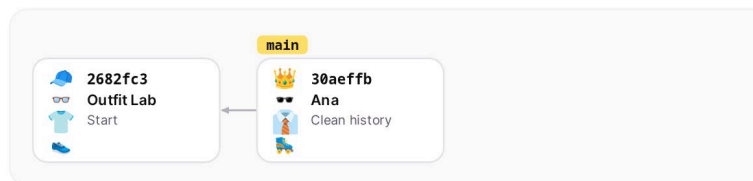
STEP 6 · CLEAN UP

The boss lab squashes every card after Start into one new card, and forces the Wall onto it. `squash + git push --force`

Outfit Lab • Lab 1 • Ana • Step 6 • The boss wants it clean

Ana - Lab 1
?

← points to the card before sticky note **YOU** your pin (HEAD): the note you're on
the Wall, last time you checked

The Wall the class's shared copy**STEP 6 · CLEAN UP** The boss lab squashes every card after Start into one new card, and forces the Wall onto it.`squash + git push --force`

STEP 6**The boss wants it clean**

Who added the boots? Where does that answer still exist?

Squash + git push --force

WHAT IT IS

cleaning up history.

WHAT IT DOES

squash turns several commits into one. Force push makes the Wall accept it anyway.

HOW GIT DOES IT

squash writes one new commit whose tree is the final state. The originals, with their authors and times, become unreachable. --force skips the fast-forward check and moves the remote branch. git gc later deletes unreachable objects.

Who first added 🦋? Wall: not found · Lab 1: not found · Lab 2: Raj (Lab 1), c7186b4 · Lab 3: Raj (Lab 1), c7186b4

Projector**STEP 6 · CLEAN UP**

Asked again, the Wall says: not found. Only the labs that kept the old cards still know.

`git push --force`**4:09****STEP 6 · CLEAN UP**

Asked again, the Wall says: not found. Only the labs that kept the old cards still know.

`git push --force`

Tuesday's paper · Just, Herzig, Czerwinka, Murphy · ISSRE 2016

Switching to Git: the Good, the Bad, and the Ugly

GOOD

Cheap branches.
Local commits and reverts.
Developers prefer a flat history.

BAD

Fast-forward forgets which branch a change came from.
Rebase rewrites the commit; the patch can change too.
Squash drops the cards, even who made them.

UGLY

Microsoft traces each change's route to main: its integration path.
For Git, that tracing had to be redesigned from scratch.
Some loss cannot be recovered.

CODE VELOCITY IN THIS CLASS

- **Lab 3** 🦋 · made 8:34 → on the Wall 8:34 · 2 s
- **Lab 1** 🦋 · made 8:34 → on the Wall 8:34 · 7 s
- **Lab 2** 🦋 · made 8:34 → on the Wall 8:35 · 31 s

After the squash, the Wall has none of these cards.

WHAT YOU LIVED

Step 3: after the fast-forward, you deleted fancy. No card says which cards were made on it.
Step 4: Rebase rewrites the change and gives it a new ID. The original card is not on the Wall.
Step 6: the squash dropped the cards, and who made them.
Step 6: the 🦋 audit failed. The Wall no longer knows who added the boots.

Flat history is data loss.

Projector**THE PAPER**

Code velocity from this class's own times. After the squash, the Wall has none of these cards.

`Just et al., ISSRE 2016`**4:18****THE PAPER**

Code velocity from this class's own times.
After the squash, the Wall has none of these cards. `Just et al., ISSRE 2016`

Outfit Lab · Teacher · 6 people · 6 online

Teacher console · Join QR · Start presenting

SCENE 18 OF 18 · PLAN 1:12

Exit question

0:04 / 3:00 · 67 min ahead

SAY

On your own. Two sentences, in the app.

DO

After a minute, show the answers. Read two aloud. Then give the answer.

ASK

A password reached the Wall. Two labs pulled. Does Undo this card (revert) remove it? If not, what would?

Show on projector

Answer

2/6 answered

Show answers on projector

with names

Ana Lab 1 · No. Revert adds a card; the old card still holds it. Change the password.

Raj Lab 1 · No, every copy still has it. Change it, then rewrite, force push and gc.

Next: What you built

Back

ON THE PROJECTOR NOW

EXIT QUESTION

A password reached the Wall. Two labs pulled. Does Undo this card (revert) remove it? If not, what would?

On your own: two sentences in the app.

EXIT QUESTION

A password reached the Wall. Two labs pulled. Does Undo this card (revert) remove it? If not, what would?

Outfit Lab · Lab 1 · Ana · Exit question

Ana · Lab 1

Outfit Lab · Lab 2 · Ben · Exit question

Ben · Lab 2

EXIT QUESTION

A password reached the Wall. Two labs pulled. Does Undo this card (revert) remove it? If not, what would?

On your own: two sentences in the app.

YOUR ANSWER

No. Revert adds a card; the old card still holds it. Change the password.

EXIT QUESTION

A password reached the Wall. Two labs pulled. Does Undo this card (revert) remove it? If not, what would?

On your own: two sentences in the app.

YOUR ANSWER

Two sentences - Enter saves

EXIT QUESTION

On their own: a password reached the Wall and two labs pulled. Does revert remove it?

4:30

EXIT QUESTION

On their own: a password reached the Wall and two labs pulled. Does revert remove it?

Outfit Lab · Lab 1 · Ana · What you built

Ana · Lab 1

My Git in 7 lines

Copy

0

Without saves, nobody knows who changed what.

1

A card is a full snapshot plus its parent.

2

A branch is a label on one card.

3

Merge compares both sides with the split card.

4

Push only moves the Wall forward.

5

Shared mistake: revert, don't reset.

6

Squash + force push: the Wall forgets who did what.

Your takeaways, in step order. Edit any line. An empty line copies the board's line.

WRAP

My Git in 7 lines: one takeaway per step, in each student's own words.

4:35

WRAP

My Git in 7 lines: one takeaway per step, in each student's own words.

Outfit Lab - Teacher6 people · 6 online

Teacher consoleJoin QRStart presenting

SCENE 19 OF 19 · PLAN 1:15

What you built0:14 / 2:00 · 70 min ahead

SAYCards never change. Sticky notes move. The Wall copies cards. That's Git. Homework: Yang et al., Sections 3.2 to 3.5. For one finding, write down what they measured.

DORead one lab's counts aloud. Point at the takeaway wall.

Export answers

Last sceneBack

Lab 12 people

Lab 22 people

Lab 32 people

ON THE PROJECTOR NOW

Cards never change. Sticky notes move. The Wall copies cards.

Next

This is the last scene.

Cards never change. Sticky notes move. The Wall copies cards.

STEP 1Git trusts my laptop for my name and the time.

STEP 2Saving moves only the note I'm on.

STEP 3No card records the branch it was made on.

STEP 5Reset is fine only if nobody has the card.

STEP 1A card is a full snapshot plus its parent.

STEP 2A branch is a label on one card.

STEP 4Push only moves the Wall forward.

STEP 6Flat history is data loss.

STEP 1Cards never change; a save makes a new card.

STEP 2Making a branch copies nothing.

STEP 4Rebase copies my cards: new IDs.

STEP 6Force push skips the safety check.

Projector

Outfit Lab · Lab 1 · Ana · What you built

Ana · Lab 1

WHAT YOU BUILT

Cards never change. Sticky notes move. The Wall copies cards.

My Git in 7 linesCopy

0Without saves, nobody knows who changed what.

1A card is a full snapshot plus its parent.

2A branch is a label on one card.

3Merge compares both sides with the split card.

4Push only moves the Wall forward.

Outfit Lab · Lab 2 · Ben · What you built

Ben · Lab 2

WHAT YOU BUILT

Cards never change. Sticky notes move. The Wall copies cards.

My Git in 7 linesCopy

0One shared draft and no history is chaos.

1Git trusts my laptop for my name and the time.

2Saving moves only the note I'm on.

3No card records the branch it was made on.

4Rebase copies my cards: new IDs.

WRAP19 scenes, 7 steps. The teacher pressed only Next, and every click ran real Git.

4:46

WRAP

19 scenes, 7 steps. The teacher pressed only Next, and every click ran real Git.